Practical Assignment Record: Digital Wallet Platform

Low-Level Design (Round 2: 30 Marks) & High-Level System Design (Round 3: 25 Marks)

PART 1: ROUND 2 -- LOW-LEVEL DESIGN (30 MARKS)

1. Aim

To implement and verify RESTful APIs for a digital wallet handling balance queries, money addition, atomic P2P transfers, transaction history, and dynamic PDF receipt generation with strict HTTP status codes and idempotency.

2. Tasks Given & Implementation Summary

* Task 1 & 2 (REST APIs & HTTP Methods): Built Flask routes with GET for retrieval and POST for state mutations. Added root endpoint GET / returning API catalog.
* Task 3 (Transfer Request & Response): Validated fromWalletId, toWalletId, positive amounts, and distinct wallets. Returns structured JSON with transaction ID and status.
* Task 4 (HTTP Status Codes): Implemented 200 (OK), 201 (Created), 400 (Bad Request), 404 (Not Found), 405 (Method Not Allowed), 409 (Conflict), 422 (Insufficient Balance), and 500.
* Task 5 (Atomic Transaction Sequence): Implemented two-phase atomic debit and credit inside database transaction. Automatic session rollback prevents partial updates.
* Task 6 (PDF Receipt Download): Built receipt_generator.py streaming official PDF receipt with SHA-256 digital signature.
* Task 7 (Testing): 15 automated pytest cases in tests/test_round2.py covering all positive and negative test cases.

3. REST API Endpoint Catalog

Method	Endpoint	Description	Status Codes
GET	/	System status, health, and available endpoints catalog	200
GET	/api/wallets/<id>/balance	Retrieve wallet balance with cache-aside lookup	200 / 404
POST	/api/wallets/<id>/add-money	Add money to wallet via payment simulation	201 / 400 / 404
POST	/api/transfers	ACID atomic P2P transfer between two distinct wallets	201 / 400 / 404 / 409 / 422
GET	/api/wallets/<id>/transactions	List full historical ledger audit entries	200 / 404
GET	/api/transactions/<id>/download	Stream downloadable official PDF transaction receipt	200 (PDF) / 404

4. Round 2 Testing & Observed Results

Automated Test Suite: tests/test_round2.py (15/15 Tests Passed Successfully in 1.4s)

Observed Results: All positive flows, 422 insufficient balances, 400 invalid inputs, 404 missing wallets, 405 method mismatches, and PDF byte streams verified.

PART 2: ROUND 3 -- HIGH-LEVEL SYSTEM DESIGN (25 MARKS)

1. Aim

To architect and demonstrate a highly scalable, fault-tolerant digital payment architecture using 10 logical microservices, database transactions, Redis Cache-Aside, asynchronous event messaging, fraud detection, and horizontal scaling.

2. 10 Logical Services Architecture

1. User Service: Manages identity, KYC status, and customer profiles.
2. Wallet Service: Authoritative balance tracking and debit/credit ledger operations.
3. Transaction Service: Coordinates ACID atomic P2P transfers with idempotency key checks.
4. Payment Service: Ingestion gateway for external payment rails (UPI, Card, NetBanking).
5. Fraud Detection Service: Evaluates velocity, anomalous spikes, and INR 100,000 transaction caps.
6. Notification Service: Asynchronously dispatches SMS, Push, and Email alerts.
7. API Gateway: Enforces TLS termination, rate limiting (Token Bucket), and JWT auth.
8. Persistent Database: PostgreSQL (production) with automatic SQLite local development fallback.
9. Cache Tier: Redis Cache-Aside with 60-second TTL and immediate invalidation on balance mutation.
10. Message Broker: Decoupled Kafka/RabbitMQ event bus ensuring notifications never block money transfers.

3. Scalability & High-Concurrency Strategy

* Horizontal Pod Autoscaling (HPA): Stateless Flask/Gunicorn worker nodes behind AWS ALB.
* Database Concurrency: Row-level locking (SELECT FOR UPDATE) prevents race conditions and double spending.
* Connection Pooling: PgBouncer handles 10,000+ client connections without resource exhaustion.
* Idempotency Guarantee: Idempotency keys stored in database prevent duplicate debit charges on network retry.
* Graceful Cache Degradation: Direct DB fallback ensures zero downtime if Redis is unavailable.

4. Round 3 Testing & Observed Results

Automated Test Suite: tests/test_round3.py (7/7 Tests Passed Successfully in 0.6s)

Observed Results: Verified idempotency replay (409 Conflict), relational persistence, cache invalidation, fraud limits rejection (INR 250k rejected), async notification dispatch, and atomic rollback guarantees.

PART 3: FINAL VERIFICATION & SUBMISSION METRICS

Deliverable / Metric	Requirement Target	Observed Status
Total Automated Tests Executed	22 Tests (pytest)	100% Passed (0 Failures)
Execution Time	< 5.0 seconds	2.06 seconds
Round 2 Low-Level Design (LLD)	30 Marks	Complete & Verified
Round 3 High-Level System Design (HLD)	25 Marks	Complete & Verified
Visual Web UI	Interactive Dashboard	Online at http://127.0.0.1:5000/ui
Transaction Receipt Engine	PDF Binary Generation	FPDF v1.7.2 with Digital Hash
Database Compatibility	PostgreSQL + SQLite	Auto-fallback Active